Recursive definitions on lists

A recursive function calls itself on a smaller problem. A base case handles an input without further recursion; a recursive case reduces the input toward that base case. Without a reachable base case or a decreasing problem, evaluation may not terminate.

A Haskell list is either empty, written [], or a head followed by a tail, written x:xs. Pattern matching separates these cases.

```
sumList [] = 0
sumList (x:xs) = x + sumList xs
```

For sumList [2,3,4], expansion gives 2 + (3 + (4 + 0)), which equals 9. The base case 0 is the identity for addition. Each recursive call consumes the tail, a strictly shorter finite list. The pattern x:xs applies only to a nonempty list.

Lists are homogeneous: their elements share a type. The empty list can be used at many element types, inferred from its context.

Original educational material / CC BY 4.0 / Bilal Amin

Building lists and choosing safe cases

The length of a finite list can be defined recursively:

```
lengthList [] = 0
lengthList (_:xs) = 1 + lengthList xs
```

The underscore ignores a value that is not needed. For [7,8,9], lengthList returns 3.

To double a list while preserving order:

```
doubleAll [] = []
doubleAll (x:xs) = (2*x) : doubleAll xs
```

The colon constructor prepends one element to a list. The expression 1 : [2,3] is [1,2,3]. The append operator ++ joins two lists; [1,2] ++ [3] is [1,2,3]. Traversing the left list makes append linear in the length of that left list.

The built-in head is partial: head [] has no valid element to return and raises an error. A safer interface returns Maybe, such as safeHead [] = Nothing and safeHead (x:_) = Just x. Maybe makes the empty case explicit in the result type.

Recursion over a finite list is linear when each step does constant work and consumes exactly one element.

Original educational material / CC BY 4.0 / Bilal Amin